1. Title

AgriSmart: Climate-Smart Agriculture Decision Support System

2. Abstract

AgriSmart is a web-based decision support application designed to help farmers and agriculture learners make informed crop and farm-management decisions. The system integrates weather-based advisories, crop recommendation logic, crop problem diagnosis, daily crop planning, and basic admin analytics in one platform.
The application is developed using React, TypeScript, Vite, Tailwind CSS, and client-side data/context architecture. It supports multilingual interaction (Marathi, Hindi, English) and role-based access (farmer, student, admin).
The project demonstrates a practical, scalable front-end architecture for smart agriculture use cases, while identifying future enhancements such as backend integration, secure authentication, and advanced predictive models.

3. Introduction

Agriculture faces uncertainty due to climate variability, pest outbreaks, water stress, and market fluctuation. Small and medium farmers often make decisions with limited real-time information. AgriSmart addresses this by offering a unified digital assistant for day-to-day agricultural decisions, including weather advisories, crop suitability checks, and crop health guidance.

4. Problem Statement

Farmers and students often lack:

1. A single platform for weather, crop planning, and problem diagnosis.
2. Localized guidance for crop selection and risk prevention.
3. Easy, multilingual, mobile-friendly decision support.

5. Objectives

1. Provide live weather insights with actionable advisories.
2. Recommend suitable crops based on user inputs (soil, season, climate range).
3. Help users identify crop problems and suggest treatment/prevention.
4. Generate daily crop care guidance and simple revenue estimation.
5. Support multilingual UX and role-based access.

6. Scope

1. Web application for educational and practical planning use.
2. Client-side operation with static crop/problem datasets.
3. Live weather via external APIs.
4. Admin-level dashboard analytics.
5. Future scope includes backend, secure auth, IoT integration, and predictive ML.

7. Development Methodology

Incremental modular development was used:

1. Base app setup and routing.
2. Authentication and language context.
3. Feature modules (dashboard, recommendation, solver, planner).
4. Admin analytics and UI improvements.
5. Build/testing/lint checks.

8. Feasibility Study

8.1 Technical Feasibility

1. Uses modern, stable frontend stack.
2. API integration is straightforward.
3. Runs in browser without heavy infrastructure.

8.2 Economic Feasibility

1. Open-source libraries reduce cost.
2. Low deployment cost on static hosting platforms.

8.3 Operational Feasibility

1. Simple UI and guided forms.
2. Language support improves adoption among local users.

9. Software and Tools

1. React 18
2. TypeScript
3. Vite
4. Tailwind CSS
5. React Router
6. TanStack Query
7. Recharts
8. Vitest + Testing Library
9. ESLint

10. System Requirements

10.1 Hardware

1. Standard laptop/desktop
2. 4 GB RAM minimum

10.2 Software

1. Node.js + npm
2. Modern browser (Chrome/Edge/Firefox)
3. VS Code (recommended)

11. Requirement Analysis

11.1 Functional Requirements

1. User registration/login/logout
2. Role-based route access
3. Weather search by city and geolocation
4. Crop database listing/search
5. Crop recommendation engine
6. Crop problem diagnosis and report download
7. Daily planner with stage/profit estimation
8. Admin analytics dashboard
9. Language switching (EN/HI/MR)

11.2 Non-Functional Requirements

1. Responsive UI
2. Usability for non-technical users
3. Maintainable modular code
4. Basic performance for dashboard and search
5. Extensible architecture for future backend

12. System Architecture

AgriSmart follows a **client-side layered architecture**:

1. Presentation Layer: React pages + reusable UI components.
2. Application Layer: Context providers (Auth, Language) + hooks.
3. Service Layer: Weather service and advisory/scoring logic.
4. Data Layer: Static datasets and localStorage persistence.

13. Module Description

13.1 Authentication Module

1. Handles login/register/logout.
2. Stores session and users in localStorage.
3. Enforces protected and admin routes.

13.2 Language Module

1. Supports Marathi, Hindi, English.
2. Loads JSON translation dictionaries.
3. Persists selected language in localStorage.

13.3 Dashboard Module

1. Displays weather metrics and advisories.
2. Provides quick navigation to feature modules.
3. Includes crop weather score and disease warning summary.

13.4 Crop Database Module

1. Displays crop cards with agronomic attributes.

2. Search by crop name.

13.5 Crop Recommendation Module

1. Accepts soil/season/temp/rainfall inputs.
2. Scores crops by overlap and suitability.
3. Returns top recommendations with confidence and reasons.

13.6 Crop Problem Solver Module

1. Filters by crop and problem type.
2. Matches symptom keywords.
3. Shows treatment, prevention, recovery time.
4. Exports text report.

13.7 Daily Planner Module

1. Calculates growth stage from sowing date.
2. Generates irrigation/fertilizer/risk guidance.
3. Estimates yield and revenue by land size.

13.8 Admin Module

1. Displays user/crop/problem analytics via charts.
2. Shows registered user list and summary metrics.

14. DFD (Textual)

14.1 Level-0 DFD

External entities: User, Admin, Weather APIs
System: AgriSmart
Flows:

1. User inputs credentials/filters/query.
2. System returns recommendations, advisories, and plans.
3. System requests weather data from external APIs.
4. Admin receives analytics and user summaries.

14.2 Level-1 DFD

1. Auth Process: input credentials -> validate local user store -> create/remove session.
2. Weather Process: city/coordinates -> weather service -> advisory generation -> dashboard.
3. Recommendation Process: input constraints -> scoring engine -> ranked crops.
4. Problem Solver Process: filters + symptom text -> match dataset -> diagnosis output.
5. Planner Process: crop + sowing date + land size -> stage/risk/profit engine -> daily plan.

15. UML (Textual)

15.1 Use Case Diagram (Actors)

1. Farmer: login, weather check, recommendation, diagnosis, planner.
2. Student: same as farmer for learning/planning.
3. Admin: all user actions + admin analytics.

15.2 Class/Entity View

1. User: id, name, email, role, createdAt
2. Crop: agronomic parameters + yield + market price
3. CropProblem: type, explanation, treatment, prevention, confidence
4. WeatherData: temperature, humidity, wind, rainfall probability, condition
5. Services: weather fetch, advisory scoring, crop-weather score

15.3 Sequence (Recommendation)

1. User submits form.
2. UI validates inputs.
3. Recommendation logic filters/scorers run.
4. Top results rendered with reasons/confidence.

16. Implementation Highlights

1. Routing and access control implemented in app-level route guards.
2. Reusable dashboard layout and UI component library used for consistency.
3. Weather service has primary+fallback provider strategy.
4. Language translations structured via JSON files.
5. Data modules are static and easy to extend.

17. Testing and Validation

17.1 Testing Types

1. Functional checks of each page workflow.
2. Build validation.
3. Lint and unit-test command execution.

17.2 Current Validation Status

1. `build`: Pass
2. `test`: Fail due to setup file path mismatch
3. `lint`: Fails with some TypeScript/ESLint errors

17.3 Sample Test Cases

Test Case	Input	Expected Result	Current Result
Login Success	Valid demo credentials	Redirect to dashboard	Pass
Protected Route	Open `/dashboard` without login	Redirect to `/login`	Pass
Crop Search	Search “wheat”	Matching crop appears	Pass
Recommendation	Soil+season input	Top suggestions with score	Pass

Problem Solver	Crop + type + description	Relevant diagnosis cards	Pass
Build	`npm run build`	Production bundle	Pass
Unit Test	`npm run test`	Tests execute	Fail (setup path)

18. Limitations

1. No backend database/API for persistent secure records.
2. Password/session handling is client-side only (not production secure).
3. Recommendation/planner modules use simplified rule logic.
4. Test setup configuration issue currently blocks automated test execution.
5. Large production JS bundle indicates optimization need.

19. Future Enhancements

1. Backend with secure authentication and database.
2. Real farm profile history and personalized advisory engine.
3. ML-based disease prediction and crop yield forecasting.
4. GIS/soil map integration.
5. Offline/PWA mode for rural low-connectivity areas.
6. IoT sensor integration (soil moisture, pH, temperature).
7. Market price API integration with region-wise recommendations.

20. Conclusion

AgriSmart successfully demonstrates a practical climate-smart agriculture support platform with modular architecture, multilingual UI, weather integration, and decision-support workflows. It is a strong prototype suitable for academic demonstration and MVP use. With backend security, testing stabilization, and advanced predictive enhancements, it can evolve into a production-ready agri advisory system.

21. Bibliography (Suggested)

1. React Documentation
2. TypeScript Documentation
3. Vite Documentation
4. Tailwind CSS Documentation
5. OpenWeather API Documentation
6. Open-Meteo API Documentation
7. FAO resources on climate-smart agriculture